A Vision for Autonomous Data Agent Collaboration (Eckmann & Binnig, CIDR 2026)

CIDR 2026 | 100 level — no background assumed summary

The problem

Organizations keep data in many separate databases that don't share a common structure. Today, answering a question that spans several of these databases usually means someone first has to merge them all into one combined database with a shared layout, which takes real engineering work.

Why it matters

Building that combined database is slow and expensive, and once it's built, you can only ask the questions its fixed layout supports. If a new, unexpected question comes up — say, about a region or category the original merge didn't plan for — you're stuck rebuilding the merge, not just asking the question.

What people do today, and why it falls short

The standard approach is to extract data from each source, transform it to match a shared format, and load it into one big database (this is called ETL). This works when there are only a few data sources that don't change often, but breaks down as the number and variety of sources grows, since every new or changing source needs its own manual integration work.

What this paper does

The authors propose skipping the merge step entirely. Instead, each data source gets its own small AI "agent" that knows what it contains and can answer questions about it. When a user asks a question, a coordinator breaks it into pieces, and the agents work together, passing partial answers back and forth, until they build the full answer — without ever needing a single shared, pre-built database. The authors build a first working version of this idea and show it can answer questions about as accurately as a normal merged database, without the cost of merging.

Takeaway: Instead of merging all data sources into one database before you can query them, let AI agents representing each source negotiate and collaborate on the fly to answer a question.

A Vision for Autonomous Data Agent Collaboration (Eckmann & Binnig, CIDR 2026)

CIDR 2026 | 200 level — working CS/ML background summary

The problem

Answering natural-language queries that span multiple heterogeneous, disparate data sources currently requires integrating those sources into a centralized database with a global schema before querying can happen.

Why it matters

ETL-based integration is costly to build and maintain, and once integrated, the set of answerable queries is constrained by the fixed schema — a query outside that schema (e.g. a region or attribute the integration never modeled) simply can't be asked without re-integrating. This gets worse as the number, heterogeneity, and rate of change of data sources increases.

Current approaches and their gap

Traditional query-by-integration builds a global schema via ETL pipelines and semantic alignment; it worked when sources grew slowly, but doesn't scale with heterogeneity and dynamism. Other multi-agent-for-data-processing ideas exist in the literature, but the authors state they are the first to design a framework specifically for collaborative query answering across disparate sources without requiring a global schema — prior multi-agent approaches still assume some form of upfront structural alignment.

What this paper does

The authors propose a network of autonomous data agents, each wrapping one data source and advertising what it knows in natural language. A coordinator receives the user's NL query, decomposes it into sub-questions, and routes them to agents, who interpret each sub-question against their local schema, generate local queries (e.g. SQL), and either answer or raise a new sub-question if they're missing information. They formalize this as the Query Collaboration Protocol (QCP) — declarative for data querying the way MCP is declarative for tool integration — and build a prototype, QCP-DB, for relational data, evaluated on a fragmented version of the BIRD text-to-SQL benchmark.

Takeaway: QCP-DB shows that LLM agents negotiating over natural-language sub-questions can match a centralized Text-to-SQL system's accuracy on multi-source queries, without ever building a unified schema.

A Vision for Autonomous Data Agent Collaboration (Eckmann & Binnig, CIDR 2026)

CIDR 2026 | 300 level — subfield-familiar summary

The problem

NL query answering across independent, disparate data sources without upfront schema integration, where sources may have no explicit foreign-key/primary-key relationships and inconsistent representations of the same real-world entities (e.g. city names in different languages or abbreviations).

Why it matters

The paper motivates this at the scale of companies with many internal or cross-company sources (e.g. medical data), where fixed ETL integration can't keep pace with new sources or ad hoc queries. The authors frame QCP as eventually enabling internet-scale, open-ended querying across autonomous agents, analogous to how the internet itself decentralized access to information.

Current methods and the specific gap

Classical data federation and mediator-based systems still rely on some form of upfront schema alignment or wrapper mapping. Other multi-agent data-processing frameworks cited by the authors (a general Data Agent architecture, and AutoTQA for tabular QA) don't specifically target collaborative query answering across disparate sources without a shared schema — the authors position QCP as the first protocol-level design for this specific setting.

This paper's approach

Agents register with a coordinator via natural-language descriptions of their data and query capabilities. On a query, the coordinator runs two-phase routing: agents self-assess relevance from a broadcast query, then the coordinator decomposes the query into a stack of NeedNodes (sub-questions with destination agents and an originator), processed in dependency order. Each agent runs a generate-feedback loop — generate SQL, execute, inspect the result and its own schema/sample values, then adapt the query or raise a new information need if there's a semantic mismatch (e.g. it needs a country-to-city mapping it doesn't have). New needs get pushed back onto the coordinator's stack and routed to other agents. A result agent then does two-phase synthesis: LLM-generated code aligns schemas and normalizes values (e.g. 'Munich'/'München'/'MUC'), then incrementally merges partial results and produces the final answer.

Results

On a fragmented BIRD benchmark (50 hand-selected multi-agent-requiring queries, 4 agents), QCP-DB with a strong model (o3 coordinator + GPT-4.1 agents) roughly matches a centralized Text-to-SQL baseline (62%) under horizontal (62%) and vertical (60%) splits, dropping more under random splits (54%); a weaker model pairing drops further (50%/42%/32%). Scaling from 4 to 7 agents under random splits degrades gracefully (54%→44%). Injecting data/schema noise (renaming, abbreviations, cross-language translation) only cost about 4 percentage points of accuracy, with translation to German having no measurable effect.

Takeaway: The paper's core bet is that a declarative, MCP-like protocol for agent-to-agent negotiation can substitute for upfront schema integration, at a modest accuracy cost that seems to hold up even under injected heterogeneity.

A Vision for Autonomous Data Agent Collaboration (Eckmann & Binnig, CIDR 2026)

CIDR 2026 | 400 level — expert / non-obvious details summary

The genuinely hard case the paper surfaces

The 'anecdote' in Section 4 is the most technically interesting result: horizontal splits (row-wise partitions) look like they should be trivial for a system like this, since it's just concatenation. But the authors show a concrete case — a Purchases/Products join split across two agents by row — where the agent holding the matching category rows can answer part of the filter locally, but the other agent holds additional matching rows it can't discover unless it's specifically asked to contribute using the join key rather than the original filter predicate. This means correct horizontal-split handling requires an agent to recognize it should contribute on a different basis than the query it was literally asked — a non-obvious failure mode that a naive per-agent SQL-generation approach would miss silently (wrong answer, not a visible error).

Query Stack / NeedNode design

The NeedNode stack makes decomposition trackable: each node carries a sub-question, candidate destination agents, and an originator, with the user's original query at the stack bottom so it resolves last. New needs raised mid-execution get pushed on top — dependency ordering is implicit in stack position, not an explicit DAG. Not addressed: how the system would behave if two independently-raised needs turned out to have a mutual dependency.

Result synthesis is doing real work, at synthesis-time not query-time

Schema/value alignment (mapping 'corp_name'/'company' to one column; normalizing 'München'/'MUC'/'Munich') is LLM-generated Python code at result-merge time, not negotiated between agents beforehand. So the Exp. 3 noise result (~4% degradation) is really a claim about this alignment step's robustness, not about routing or decomposition robustness — the vertical split used for that experiment is the authors' own 'most challenging scenario for result alignment.'

Evaluation caveats worth flagging

50 hand-selected BIRD queries chosen specifically because they need multi-agent coordination — reasonable, but the reported accuracy describes a curated slice, not BIRD broadly. The paper states most BIRD queries remain answerable by a single partition even after fragmentation, somewhat undercutting the premise that real queries need collaboration as often as the framing implies. The coordinator is currently centralized (distributing it is future work), so the internet-scale vision is further away than the working prototype.

Positioning vs. closest prior work

The novelty claim is specifically about being first to design a protocol for collaborative query answering without a global schema — not first to apply multi-agent LLM systems to data tasks generally (a Data Agent architecture paper and AutoTQA are cited as adjacent prior work). QCP-vs-MCP (declarative data querying analogous to declarative tool integration) is the paper's central framing for the contribution.

Takeaway: The strongest part of this paper is the horizontal-split anecdote, which honestly surfaces a real, non-obvious failure mode rather than hiding it; the weakest part is that the evaluation is curated toward cases the system should do well on, and the coordinator's centralization means the 'internet-scale' vision in the roadmap is aspirational, not demonstrated.